Google Agent Development Kit (ADK)

Source: google.github.io/adk-docs · Google Developers Blog · ADK GitHub

Version: ADK Python (latest) · ADK 2.0 (graph-based workflows)

Why this matters: ADK is the framework built by the team you are interviewing with. Understanding ADK's architecture means you can connect every agent paper you've read to how Google actually builds production agent systems.

Contents

1 Why ADK Exists **3**

2 Core Architecture **3**
- 2.1 The Fundamental Components . . . 3
- 2.2 Agent Types . . . 3
  - 2.2.1 LlmAgent . . . 3
  - 2.2.2 Workflow Agents . . . 4
- 2.3 The Runner . . . 4

3 Sessions and State **5**
- 3.1 Session . . . 5
- 3.2 State . . . 5
- 3.3 Memory . . . 6
- 3.4 Session Storage Backends . . . 6

4 Tools **6**
- 4.1 Function Tools . . . 6
- 4.2 MCP Tools . . . 7
- 4.3 Agent as Tool (AgentTool) . . . 7

5 Callbacks **8**

6 Multi-Agent Systems in ADK **8**
- 6.1 Agent Hierarchy . . . 8
- 6.2 Agent Transfer . . . 9
- 6.3 A2A Integration . . . 9

7 Deployment **9**

8 ADK vs LangGraph **9**

9 ADK in the Context of Agent Papers **10**

10 Self-Check Questions **10**
- 10.1 Core Concepts . . . 10
- 10.2 Session and State . . . 10
- 10.3 Tools . . . 11
- 10.4 Architecture . . . 11

11 Interview Questions **11**
- 11.1 Foundational . . . 11

11.2 Architecture 11
11.3 Production 11
11.4 Deep Dive 11

Why ADK Exists

Google Cloud AI needed a framework that could:

1. Build agents that work in production enterprise environments (not just demos)
2. Natively support both MCP (for tools) and A2A (for agent-to-agent coordination)
3. Deploy to Google Cloud infrastructure (Agent Engine, Cloud Run, GKE)
4. Work with any LLM — optimized for Gemini, but model-agnostic

The design philosophy: ADK makes agent development feel like software development. Agents are defined as code, testable, evaluatable, observable, and deployable through standard software engineering practices.

ADK is:

- **Model-agnostic** — supports Gemini, Claude, Llama, Mistral, any LiteLLM-compatible model
- **Deployment-agnostic** — local, Cloud Run, GKE, Agent Engine
- **Framework-compatible** — interoperates with LangGraph, CrewAI, and AutoGen via A2A

Core Architecture

The Fundamental Components

Listing 1: ADK Component Overview

```
+-------------------------------------------------------+
|                    ADK Application                     |
|                                                        |
|  +-------------+   +---------+   +---------------+   |
|  |    Agent    |-->|  Runner |-->|    Session    |   |
|  | (LLM / WF)  |   | (engine)|   |   + State     |   |
|  +------+------+   +---------+   +---------------+   |
|         |                                              |
|  +------+------+   +---------+   +---------------+   |
|  |    Tools    |   | Memory  |   |   Callbacks   |   |
|  |(fn/MCP/A2A) |   |(cross-  |   |   (hooks)     |   |
|  +-------------+   |session) |   +---------------+   |
|                    +---------+                         |
+-------------------------------------------------------+
```

Agent Types

ADK has two fundamental agent categories:

LlmAgent

Uses a language model for reasoning and decision-making. The LLM decides what to do next — which tool to call, whether to transfer to another agent, or how to respond. Dynamic and adaptive.

Listing 2: LlmAgent Definition

```python
from google.adk.agents import LlmAgent
```

```
research_agent = LlmAgent(
    name="research_agent",
    model="gemini-2.0-flash",
    instruction="You are a research specialist. Search the web and synthesize
        information.",
    tools=[google_search_tool, web_fetch_tool],
)
```

Workflow Agents

Deterministic controllers — they do not use an LLM for routing decisions. They follow a fixed execution pattern.

Listing 3: Three Workflow Agent Types

```
from google.adk.agents import SequentialAgent, ParallelAgent, LoopAgent

# Sequential: step1 -> step2 -> step3 (always in order)
pipeline = SequentialAgent(
    name="pipeline",
    sub_agents=[data_fetch_agent, analysis_agent, report_agent]
)

# Parallel: all sub-agents run simultaneously, wait for all to finish
research_team = ParallelAgent(
    name="research_team",
    sub_agents=[web_agent, database_agent, arxiv_agent]
)

# Loop: runs sub-agent repeatedly until condition is met
refinement_loop = LoopAgent(
    name="refine",
    sub_agents=[draft_agent],
    max_iterations=5
)
```

When to use which:

- Use `LlmAgent` when routing logic is complex or data-dependent
- Use Workflow agents when execution pattern is known and deterministic
- Combine them: an `LlmAgent` at the top routing to `SequentialAgent` sub-pipelines

The Runner

The **Runner** is the execution engine. It orchestrates the event loop, manages agent invocations, coordinates with session storage, and handles the flow of Events through the system.

Listing 4: Runner Usage

```
from google.adk.runners import Runner

runner = Runner(
    agent=my_agent,
    app_name="my_app",
    session_service=session_service,
)

async for event in runner.run_async(
```

```
    user_id="user_123",
    session_id="session_abc",
    new_message=user_input
):
    process_event(event)
```

The event loop: Every interaction is a stream of Events. The Runner receives a user message → creates an Event → passes to agent → receives agent response Events (may include tool calls) → executes tools → feeds results back as Events → continues until final response.

Sessions and State

Most important section for your interview. ADK's session system directly maps to the JD's "managing session persistence in stateful applications." Be able to explain every component here.

Session

A **Session** represents a single conversation thread between a user and an agent.

Listing 5: Session Structure

```
{
  "session_id": "sess_abc123",
  "user_id": "user_456",
  "app_name": "research_app",
  "events": [
    Event(author="user",  content="Research climate change"),
    Event(author="agent", content="I'll research this for you"),
    Event(author="tool",  content={"search_results": [...]}),
    Event(author="agent", content="Here are the findings..."),
  ],
  "state": {
    "research_topic": "climate change",
    "sources_found": 12,
    "current_step": "synthesis"
  }
}
```

State

State is the agent's working memory within a session — a key-value store for data the agent needs to maintain across multiple turns. State is separate from Events (conversation history). Events are immutable records; State is mutable working data.

Listing 6: State Read and Write

```
# Reading state in a tool/callback
topic = tool_context.state.get("research_topic", "general")

# Writing state
tool_context.state["sources_found"] = 15
tool_context.state["last_search_query"] = "climate impacts 2025"

# User-scoped state (persists across sessions)
tool_context.state["user:preferred_language"] = "Korean"

# App-scoped state (shared across all users)
```

```
tool_context.state["app:global_counter"] = 100
```

State scoping:

Scope	Prefix	Lifetime
Session state	(default, no prefix)	Current session only
User state	`user:`	Persists across sessions for same user
App state	`app:`	Shared across all users

Memory

Memory is distinct from State. It is long-term storage that persists across multiple sessions and can be searched semantically. While State is the agent's scratchpad for the current conversation, Memory is the agent's long-term knowledge base about a user.

Listing 7: Memory Service Options

```
from google.adk.memory import InMemoryMemoryService, VertexAiRagMemoryService

# Development / testing (lost on restart)
memory_service = InMemoryMemoryService()

# Production: Vertex AI RAG (semantic search across past sessions)
memory_service = VertexAiRagMemoryService(
    rag_corpus="projects/.../ragCorpora/..."
)
```

Component	Use case	Example
State	Current conversation working data	"What step are we on?"
Memory	Cross-session user knowledge	"What topics has this user asked about?"
Events	Immutable conversation history	"What was said in this session?"

Session Storage Backends

Backend	Use Case
`InMemorySessionService`	Development, testing — lost on restart
`DatabaseSessionService`	Production — persists to SQL database
`VertexAiSessionService`	Production GCP — managed service

AgentLink mapping: Redis (hot session state) ≈ InMemory-speed with persistence. PostgreSQL (checkpoint) ≈ DatabaseSessionService durability. The ADK backend abstraction validates this two-tier design.

Tools

Function Tools

Listing 8: Function Tool — The docstring IS the LLM description

```python
from google.adk.tools import FunctionTool

def search_database(query: str, limit: int = 10) -> dict:
    """Search the internal database for relevant records.

    Args:
        query: The search query string
        limit: Maximum number of results to return

    Returns:
        Dictionary containing search results
    """
    results = db.search(query, limit=limit)
    return {"results": results, "count": len(results)}

search_tool = FunctionTool(search_database)
```

Critical: The docstring IS the tool description shown to the LLM. Write it clearly and precisely — it determines how and when the LLM decides to use the tool.

MCP Tools

ADK has native MCP client integration — connect any MCP server and use its tools as if they were native ADK tools:

Listing 9: MCP Tool Integration

```python
from google.adk.tools.mcp_tool import MCPToolset, StdioServerParameters

mcp_tools = MCPToolset(
    connection_params=StdioServerParameters(
        command="python",
        args=["./my_mcp_server.py"]
    )
)

agent = LlmAgent(
    name="agent_with_mcp",
    tools=[mcp_tools],  # MCP tools alongside native tools
)
```

Agent as Tool (AgentTool)

Any agent can be used as a tool by another agent:

Listing 10: AgentTool for Composable Architectures

```python
from google.adk.tools import AgentTool

specialist_as_tool = AgentTool(agent=specialist_agent)

orchestrator = LlmAgent(
    name="orchestrator",
    tools=[specialist_as_tool, other_tools]
)
```

AgentTool vs A2A: AgentTool is for same-process agent invocation (tightly coupled, same codebase). A2A is for cross-process, cross-framework, cross-vendor agent communication (loosely coupled, different teams).

Callbacks

Callbacks are custom code that runs at specific execution points. They enable observability, guardrails, caching, and behavior modification without changing the agent's core logic.

Listing 11: Callback Execution Points

```
Before Agent Run  --> agent_callback (before_agent_callback)
  |
  |--> Before Model Call --> model_callback
  |        |
  |        +--> [LLM generates response]
  |
  |--> After Model Call --> model_callback
  |
  |--> Before Tool Execution --> tool_callback
  |        |
  |        +--> [Tool executes]
  |
  \--> After Tool Execution --> tool_callback

After Agent Run --> agent_callback (after_agent_callback)
```

Callback use cases:

Listing 12: Three Common Callback Patterns

```python
# 1. Guardrail: block dangerous tool calls
def safety_check(callback_context, tool_call):
    if contains_pii(tool_call.arguments):
        return ToolResult(error="PII detected -- blocked")
    return None  # None = proceed normally

# 2. Caching: skip re-execution of expensive tools
def caching_callback(callback_context, tool_call):
    cache_key = hash(str(tool_call))
    if cache_key in cache:
        return cache[cache_key]
    return None  # Not cached, proceed with execution

# 3. Observability: log every step
def logging_callback(callback_context, event):
    logger.info(f"Step: {event.type}, content: {event.content}")
    return None  # Always proceed
```

Multi-Agent Systems in ADK

Agent Hierarchy

Listing 13: Building a Multi-Agent System

```python
# Specialist leaf agents
web_search     = LlmAgent(name="web_search",     tools=[search_tool])
data_analysis  = LlmAgent(name="data_analysis",  tools=[analysis_tool])
report_writer  = LlmAgent(name="writer",         tools=[writing_tool])

# Orchestrator manages specialists
orchestrator = LlmAgent(
    name="orchestrator",
    instruction="Coordinate research, analysis, and writing tasks",
    sub_agents=[web_search, data_analysis, report_writer]
)
```

Agent Transfer

When an `LlmAgent` has `sub_agents`, it can transfer control to them. The LLM decides when and which sub-agent to activate based on the task context. No explicit routing code needed.

A2A Integration

Listing 14: Connecting to Remote A2A Agents

```
from google.adk.a2a import RemoteA2aAgent

# Connect to any A2A-compliant agent, regardless of framework
remote_specialist = RemoteA2aAgent(
    agent_card_url="https://specialist.example.com/.well-known/agent-card.
        json"
)

orchestrator = LlmAgent(
    name="orchestrator",
    sub_agents=[remote_specialist, local_agent]
)
# Orchestrator treats remote A2A agents exactly like local sub-agents
```

Deployment

Option	Description
Local (`adk web/run`)	Development. Web UI or CLI.
Agent Engine	Managed GCP runtime. Handles scaling, session persistence, monitoring.
Cloud Run	Serverless. Containerized deployment. Variable traffic.
GKE	Kubernetes. Full control. High-traffic production.

ADK vs LangGraph

Dimension	ADK	LangGraph
Graph model	Hierarchical agents (parent/child)	Explicit DAG with nodes and edges
Routing	LLM-driven or deterministic WF	Conditional edges, can be LLM-driven
A2A support	Native first-class	Via custom integrations
MCP support	Native first-class	Via LangChain tools
Deployment	Tightly integrated with GCP	Cloud-agnostic
Session storage	Multiple backends + managed	Custom checkpointers
Streaming	Native via Gemini Live API	Via streaming callbacks
Evaluation	Vertex AI GenAI Evaluation	LangSmith

ADK advantage: End-to-end integration — build, test, deploy, monitor, evaluate all in one framework that natively speaks MCP and A2A.

LangGraph advantage: Framework-agnostic, more explicit graph control, larger community, cloud-agnostic.

ADK in the Context of Agent Papers

Paper Concept	ADK Equivalent
ReAct (Thought-Action-Observation)	`LlmAgent` with tools — the LLM's reasoning loop
Tool use	`FunctionTool`, `MCPToolset`, `AgentTool`
Memory (MemGPT hierarchical)	`Session State` (short-term) + `Memory Service` (long-term)
Skill library (Voyager)	Tools registered on an agent, discoverable via MCP
Role-based agents (MetaGPT)	Named `LlmAgent` with specific `instruction` prompts
Multi-agent collaboration (AutoGen)	ADK multi-agent with `sub_agents` + A2A for cross-framework
Agent evaluation (AgentBench)	ADK Evaluation with Vertex AI GenAI Evaluation

The critical insight for your interview: ADK is the productionized implementation of everything the agent papers describe theoretically. When you've read ReAct and understood the Thought-Action-Observation loop, you can recognize it in `LlmAgent`'s execution. When you've read MemGPT, you can explain ADK's Session/State/Memory hierarchy. This connection between theory and production implementation is what makes a strong candidate.

Self-Check Questions

Core Concepts

What is the difference between an `LlmAgent` and a Workflow Agent? When would you use each?

What are the three Workflow Agent types and what execution pattern does each implement?

What does the Runner do? Why is it a separate component from the Agent?

What is an Event in ADK? Give three examples of things that generate Events.

Session and State

What is the difference between Session State and Memory in ADK?

What are the three state scopes (session, user, app) and when would you use each?

How would you implement a feature where an agent remembers a user's preferences across multiple conversations?

How does ADK's Session concept map to your AgentLink session persistence design?

Tools

What is the difference between AgentTool and A2A integration? When would you choose one over the other?

Why is the docstring of a function tool so important?

How does ADK's MCPToolset work? What does it enable?

Architecture

How does agent transfer work in a hierarchical ADK system?

How would you implement observability for an ADK agent in production?

Compare ADK's approach to session persistence with how you would implement it in AgentLink.

Interview Questions

Foundational

- "What is Google ADK and how does it relate to A2A and MCP?" → ADK is the framework that uses both protocols natively
- "What types of agents does ADK support and when would you use each?"
- "How does ADK handle session state and long-term memory? What's the difference?"

Architecture

- "You're building a research pipeline with web search, analysis, and writing stages. How would you structure this in ADK?" → SequentialAgent or LlmAgent with sub-agents, explain tradeoffs
- "How would you make an ADK agent communicate with a LangGraph agent?" → A2A protocol, RemoteA2aAgent
- "How does ADK's AgentTool differ from A2A remote agent integration?"

Production

- "How would you deploy an ADK agent to handle 10,000 concurrent users?" → Cloud Run with auto-scaling, VertexAI Session Service, stateless agent design
- "How would you evaluate whether your ADK agent is performing correctly?" → ADK Evaluation framework, test cases with expected trajectories
- "What observability would you add to a production ADK deployment?" → Cloud Trace, Event logging, Callbacks for metrics

Deep Dive

- "Walk me through what happens when a user sends a message to an ADK LlmAgent with three tools." → Event creation → Runner → LLM inference → tool call decision → tool execution → result Event → next inference → final response

- "How does ADK's callback system enable you to add guardrails without modifying agent code?"
- "How would you implement caching for expensive tool calls in ADK?"

Official Docs: https://google.github.io/adk-docs/
Technical Overview: https://google.github.io/adk-docs/get-started/about/
Sessions & Memory: https://google.github.io/adk-docs/sessions/
ADK GitHub (Python): https://github.com/google/adk-python